

Final Project Sprint Feedback

Recommendations, Technical Direction, Data Samples, and Next Steps

Guidance on when to use Machine Learning, SQL/Pandas, LLMs, RAG, Cloud LLMs, or local models

Prepared for student project follow-up

Table of Contents

- 1. General Decision Framework: ML, BI/SQL, LLM, RAG, Fine-Tuning, Cloud vs Local
- 2. Michael - Board Game Recommendation System
- 3. Ayse - Online Shopper Purchase Intention Prediction
- 4. Siyam - FAA SDR Aviation Failure Pattern Mining
- 5. Barbara and Nuneeb - Banking Transactions and Personal Spending Assistant
- 6. Eraldo - Car Export Opportunity and Arbitrage Analyzer
- 7. AI Business Assistant / WhatsApp Ordering System - Group Not Specified
- 8. Local AI with Private Documents / Dental Office Use Case - Group Not Specified
- 9. Customer Churn Prediction - Group Not Specified
- 10. Government Funding Search Assistant - Group Not Specified
- 11. Common Sprint Review Questions and Final Reminders

1. General Decision Framework

This document summarizes the technical feedback given during sprint follow-up discussions. The goal is to help each student or team clarify the business problem, the input data, the expected output, and the correct role of Machine Learning, SQL/Pandas, LLMs, RAG, cloud LLMs, and fine-tuning.

Key principle

Do not start with the tool. Start with the business question, the data type, and the expected output. Then choose the right technique.

1.1 When to use each technique

Technique	Use it when...	Do not use it when...	Example
SQL / Pandas / DuckDB	The data is structured and exact calculations are required.	The answer depends on long documents or policies.	Monthly spending, category totals, profit calculation, product filtering.
Supervised ML	You have labeled historical data and want to predict a target variable.	There is no label or the goal is only to retrieve text.	Purchase prediction, churn prediction, risk prediction.
Recommendation algorithms	The system must rank or suggest items to users.	The system only summarizes documents.	Board game recommendations using content-based filtering or collaborative filtering.
Association rule mining	The goal is to discover frequent co-occurring patterns.	You only need a single prediction label.	FAA SDR failure patterns using Apriori or FP-Growth.
LLM	You need natural language understanding, explanation, translation, summarization, or conversational interaction.	You need exact totals, official calculations, or deterministic transactions.	Explain a prediction, normalize a misspelled product name, answer in the user language.
RAG	The system must answer from documents, policies, PDFs, manuals, rules, FAQs, or private knowledge bases.	The data is purely structured and the task is calculation, classification, ranking, or database filtering.	Local AI answering from clinic policies; FAQ support; import/export regulations.
Fine-tuning	You need to teach tone, style, output format, or a repeated task behavior.	You need fresh, changing facts or current rules.	A model that always explains BI results in a fixed style.
Cloud LLM	Local hardware is too weak or the app needs reliable hosted inference.	Raw private data would be sent without anonymization or consent.	Send only anonymized summaries to a cloud LLM for explanation.
Local LLM	Privacy is critical and the machine/server can run the model.	The machine lacks RAM/VRAM or performance is too low.	Local RAG over private clinic documents.

1.2 Generic architecture patterns

Problem type	Recommended architecture
Structured transaction analysis	CSV / SQL / Pandas / DuckDB -> calculations -> LLM explanation.
Supervised prediction	Structured data -> train/test split -> ML model -> metrics -> LLM explanation.
Recommendation system	Structured item/user data -> recommender algorithm -> ranked results -> LLM explanation.
Document question answering	Documents -> chunking -> embeddings -> retrieval -> LLM answer with sources.
Transactional chatbot	LLM intent detection -> database lookup -> backend logic -> confirmation response.
Regulation or policy assistant	Official documents or rules -> RAG -> LLM answer with citations/sources.

2. Michael - Board Game Recommendation System

Main question: Which board game should we recommend to a user, and why?

Michael, the project should focus on building a board game recommendation system, not on RAG as the main solution. The data is structured: games, mechanics, themes, categories, ratings, complexity, play time, number of players, and user ratings. The LLM can be added after the recommender algorithm to explain the recommendation in natural language.

Component	Recommendation for this project
Business problem	Help a user choose the best board game based on preferences and/or similar users.
Input data	BoardGameGeek-style structured data: title, category, mechanics, themes, complexity, play time, rating, user ratings.
Output	Ranked list of recommended games with explanations.
Core technique	Content-based filtering, collaborative filtering, or hybrid recommender system.
LLM?	Yes, after recommendation, to explain why a game is recommended.
RAG?	Not as the core. Only useful if adding rulebooks, reviews, forum posts, or long game descriptions.
ML model?	Possible, but recommendation algorithms are the first priority.

2.1 Content-based filtering vs collaborative filtering

Approach	Logic	Board game example
Content-based filtering	Recommend items similar to what the user likes based on item features.	If the user likes cooperative strategy games with medium complexity and 60-90 minute play time, recommend games with similar mechanics and complexity.
Collaborative filtering	Recommend items based on what similar users liked.	If users who liked Pandemic and Catan also liked Carcassonne, recommend Carcassonne to a similar user.
Hybrid recommender	Combine item similarity and user rating patterns.	Rank games using both game features and user behavior.

2.2 Steps for Michael

1. **Define the objective** - Answer: Which board game should I recommend to this user, and why?
2. **Understand the dataset** - Identify useful columns: title, category, mechanics, themes, play time, players, complexity, rating, rating count, and user ratings.
3. **Clean the data** - Remove duplicates, handle missing values, normalize categories/mechanics, and keep useful columns.
4. **Build a content-based recommender** - Use game features to calculate similarity between games and user preferences.
5. **Add collaborative filtering if possible** - Use user ratings to recommend games liked by similar users.
6. **Compare approaches** - Explain whether content-based, collaborative filtering, or hybrid works best.
7. **Add LLM explanation** - Use the LLM to explain the recommendation after the algorithm ranks the games.
8. **Avoid RAG as core** - Use RAG only if the project includes rulebooks, long reviews, or forum discussions.
9. **Build the demo** - Create a Streamlit interface where the user enters preferences and receives recommended games.
10. **Evaluate the system** - Use examples, similarity quality, rating patterns, precision@k, or top-k evaluation where possible.

2.3 Sample structured data

game_id	title	mechanics	complexity	play_time	avg_rating	ratings_count
1	Pandemic	Cooperative; Hand Management	2.4	45	7.6	120000
2	Catan	Trading; Network Building	2.3	90	7.1	115000
3	Azul	Pattern Building; Drafting	1.8	45	7.8	90000
4	Terraforming Mars	Engine Building; Card Drafting	3.2	120	8.4	95000

Final pipeline: BoardGameGeek structured data -> data cleaning -> recommender algorithm -> ranked games -> LLM explanation -> Streamlit demo

3. Ayse - Online Shopper Purchase Intention Prediction

Main question: Can we predict whether an online visitor will make a purchase based on browsing behavior?

Ayse, your project is a supervised machine learning classification problem. The target variable is Revenue. Revenue = TRUE means the visitor made a purchase, and Revenue = FALSE means the visitor did not make a purchase. RAG is not necessary because the dataset is structured and already contains a labeled target column.

Component	Recommendation for this project
Business problem	Identify visitors who are likely to purchase before they leave the website.
Input data	Online shopper behavior: page visits, page duration, bounce rate, exit rate, visitor type, traffic type, month, weekend, page value.
Output	Purchase prediction: Revenue TRUE/FALSE or purchase probability.
Core technique	Supervised classification.
LLM?	Yes, after the model prediction, to explain the result in business language.
RAG?	No for the core dataset. Only useful if adding marketing policies, product descriptions, or campaign rules.
ML model?	Yes: Logistic Regression, Random Forest, XGBoost, LightGBM, SVM baseline.

3.1 Why not just use an LLM?

We should not use only an LLM because the main task is a supervised machine learning prediction problem. The dataset contains structured features such as page duration, bounce rate, exit rate, visitor type, month, and the target variable Revenue, so the system must learn patterns from historical labeled data. A machine learning model can be trained, tested, evaluated, and measured using accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix. An LLM alone may produce a convincing explanation, but it does not reliably learn from the dataset, calculate probabilities, or prove its performance with evaluation metrics. The best architecture is: the ML model predicts the outcome, and the LLM explains the prediction in simple business language.

3.2 Steps for Ayse

- Define the objective** - Predict whether an online visitor will make a purchase.
- Identify the target variable** - Use Revenue as the target: TRUE means purchase, FALSE means no purchase.
- Identify input features** - Use Administrative, ProductRelated, BounceRates, ExitRates, PageValues, Month, VisitorType, Weekend, TrafficType, Browser, Region, and related columns.
- Clean and prepare data** - Check missing values, encode categorical columns, and convert Revenue to 1/0.
- Split the data** - Use train/test split, for example 80/20.
- Train classification models** - Start with Logistic Regression and Random Forest, then compare with XGBoost or LightGBM if possible.
- Evaluate models** - Do not rely on accuracy only. Use confusion matrix, precision, recall, F1-score, and ROC-AUC.
- Explain important features** - Show which variables influence purchase prediction.
- Add LLM explanation** - Use the LLM to explain the ML prediction in business language.
- Build the demo** - Create a Streamlit dashboard with upload, prediction, metrics, and LLM explanation.

3.3 Feedback on current work

Ayse, your Random Forest ROC curve and AUC score are very good progress. An AUC around 0.921 is strong because it indicates that the model separates purchasing and non-purchasing visitors well. However, you should not stop at AUC. Add the confusion matrix, precision, recall, F1-score, feature importance, and a short business interpretation. Also verify that the train/test split is correct and that there is no data leakage, especially from variables that may be too close to the purchase outcome.

3.4 Sample data

Administrative	ProductRelated	BounceRates	ExitRates	PageValues	Month	VisitorType	Weekend	Revenue
0	12	0.02	0.05	0.00	May	Returning_Visitor	False	False
2	45	0.00	0.01	35.40	Nov	Returning_Visitor	True	True
1	8	0.12	0.25	0.00	Feb	New_Visitor	False	False

Final pipeline: structured CSV data -> data cleaning -> supervised ML classification -> model evaluation -> LLM explanation -> Streamlit dashboard

4. Siyam - FAA SDR Aviation Failure Pattern Mining

Main question: What combinations of failure descriptions and aircraft components frequently co-occur in recent FAA Service Difficulty Reports?

Siyam, your project is strong because it uses a real aviation safety dataset and focuses on discovering hidden failure patterns. The core method is not RAG. The core method should be text preprocessing plus association rule mining using Apriori or FP-Growth with support, confidence, and lift. RAG can be added later to retrieve original reports that support a discovered rule.

Component	Recommendation for this project
Business problem	Help aviation safety analysts discover recurring failure patterns in maintenance reports.
Input data	FAA SDR reports from the last three years, including free-text discrepancy and structured codes.
Output	Association rules with support, confidence, lift, and aviation interpretation.
Core technique	Text mining and association rule mining.
LLM?	Optional for explanation and summarization after rules are discovered.
RAG?	Not core. Useful only to retrieve original SDR reports supporting a rule.
ML model?	Not a classical supervised model. Use Apriori / FP-Growth for pattern discovery.

4.1 Feedback on current work

Siyam, exploring the 2023, 2024, and 2025 datasets separately before combining them is a very good step because column names or structures may change from one year to another. Combining the data into a large dataset with many rows and columns gives a strong base for the project. Your work documenting the meaning of FAA codes is also excellent, because rules become interpretable only when raw codes are translated into readable aviation concepts such as fluid loss, smoke, flame, warning indication, or vibration.

4.2 Steps for Siyam

- Select useful columns** - Focus on Discrepancy, NatureOfCondition, component code, aircraft make/model, date, corrective action, and related fields.
- Clean raw text** - Lowercase, remove punctuation, remove special characters, remove extra spaces, and standardize formats.
- Remove stop words** - Remove common words with no technical meaning.
- Normalize aviation terms** - Convert similar expressions such as hydraulic fluid leakage and leaking hydraulic line into hydraulic leak.
- Decode structured codes** - Use the FAA code dictionary to convert K, J, B, and other codes into readable meanings.
- Extract meaningful items** - Create items such as fluid loss, warning indication, smoke, vibration, electrical fault, landing gear, engine issue.
- Prepare transaction format** - Each SDR report becomes a transaction containing clean items.
- Run Apriori or FP-Growth** - Find frequent combinations of failure terms and components.
- Evaluate rules** - Use support, confidence, and lift to identify meaningful rules.
- Interpret results** - Explain what the rules mean for aviation maintenance and safety.

4.3 Sample transformation

Raw SDR text / code	Cleaned item
HYDRAULIC FLUID LEAK FOUND NEAR LANDING GEAR LINE	hydraulic leak; landing gear; fluid loss
Code K	fluid loss
Code J	warning indication
SMOKE REPORTED IN AVIONICS BAY	smoke; avionics; electrical issue

4.4 Example association rules

Rule	Support	Confidence	Lift	Interpretation
IF hydraulic leak + landing gear THEN unscheduled maintenance	3.8%	72%	2.4	Reports mentioning hydraulic leak and landing gear are strongly associated with unscheduled maintenance.

Rule	Support	Confidence	Lift	Interpretation
IF smoke + avionics THEN electrical inspection	1.6%	68%	2.1	Smoke in avionics-related reports often leads to electrical inspection.
IF crack detected + fuselage THEN structural repair	2.2%	74%	2.8	Structural cracks in fuselage-related reports often result in repair actions.

Final pipeline: FAA SDR reports -> text cleaning -> term normalization -> item extraction -> transaction format -> Apriori/FP-Growth -> association rules -> interpretation with support, confidence, and lift

5. Barbara and Nuneeb - Banking Transactions and Personal Spending Assistant

Main question: How can a user understand and improve their spending habits using banking transaction data?

Barbara and Nuneeb, since the project focuses only on banking transactions, it is mainly a BI / SQL / Pandas / DuckDB analytics project with an LLM explanation layer. It is not mainly RAG and it does not need fine-tuning for the current stage. The system should calculate all numbers locally or in the backend, then use the LLM to explain the results.

Component	Recommendation for this project
Business problem	Help students, families, and professionals understand spending, detect unusual transactions, and monitor budgets.
Input data	Bank CSV transactions from RBC, Scotiabank, TD, BMO, CIBC, or simulated/normalized transaction files.
Output	Monthly totals, category totals, top merchants, alerts, charts, and LLM explanations.
Core technique	SQL / Pandas / DuckDB analytics and BI-style calculations.
LLM?	Yes, for explanation, natural language insights, multilingual support, and profile-based guidance.
RAG?	Not core. Only useful if adding PDF statements, bank policies, or documents later.
Fine-tuning?	Not needed. It is not the right tool for calculations or changing knowledge.
Cloud LLM?	Acceptable if local machine lacks memory, but send only anonymized summaries, not raw banking CSV.

5.1 Three user profiles

Profile	Typical concerns	Example explanation
Student	Food, transport, subscriptions, small recurring expenses, limited monthly income.	Your subscription expenses are high compared to your total monthly income.
Family	Groceries, rent/mortgage, utilities, children expenses, insurance, monthly budget stability.	Groceries and utilities are your largest recurring categories this month.
Professional	Income, business expenses, restaurants, travel, subscriptions, savings capacity.	Your restaurant and travel expenses increased compared to last month.

5.2 Steps for Barbara and Nuneeb

31. **Define the input data** - Use banking transactions with date, merchant, description, category, amount, transaction_type, payment_method, and bank_name.
32. **Normalize bank CSV formats** - RBC, Scotiabank, TD, BMO, and CIBC may export different column names. Convert all files to one common schema.
33. **Clean the transactions** - Fix missing values, standardize dates, clean merchant names, normalize categories, and separate income from expenses.
34. **Build analytics** - Calculate monthly totals, category totals, merchant spending, income vs expenses, repeated subscriptions, and unusual transactions.
35. **Add alerts and simple forecasting** - Detect overspending, high-value transactions, repeated subscriptions, and categories increasing compared to previous months.
36. **Use the LLM for explanation only** - The LLM explains calculated results in simple language. It does not invent numbers or calculate totals by itself.
37. **Use a cloud LLM carefully if needed** - If the local machine has limited memory, use a cloud LLM but send only anonymized summaries, never the full raw banking CSV.
38. **Build the demo** - Create a Streamlit dashboard with CSV upload, charts, indicators, profile selection, and LLM explanation.

5.3 Cloud LLM guidance

Using a cloud LLM is reasonable if the local machine does not have enough memory. However, sensitive banking data must not be sent in raw form. The best approach is to calculate all results locally with SQL, Pandas, or DuckDB, then send only anonymized summaries to the cloud LLM for explanation. For a student demo, a provider with a free tier

may be enough, but the team must check current usage limits and avoid enabling paid billing without understanding the cost.

Local calculation:

Total expenses = 980

Groceries = 280

Subscriptions = 95

Alert = subscriptions increased by 30%

Cloud LLM receives only:

User profile: student. Total expenses: 980. Groceries: 280. Subscriptions: 95. Subscriptions increased by 30% compared to last month.

5.4 Sample normalized data

date	merchant	category	amount	transaction_type	payment_method	bank_name
2026-05-03	Metro	Groceries	-84.50	expense	debit	RBC
2026-05-06	STM	Transport	-97.00	expense	debit	Scotiabank
2026-05-10	Employer Payroll	Income	1800.00	income	deposit	TD
2026-05-14	Netflix	Subscriptions	-17.99	expense	credit	BMO

Final pipeline: bank CSV files -> data normalization -> SQL/Pandas/DuckDB analytics -> profile-based insights
-> anonymized summary -> cloud or local LLM explanation -> Streamlit dashboard

6. Eraldo - Car Export Opportunity and Arbitrage Analyzer

Main question: Which car should be exported, to which country, for which buyer, with what expected profit and risk?

Eraldo, the project should not remain vague with the idea that Canada exports cars. The stronger project is a decision-support system for car export opportunities. The system should evaluate buyer type, destination market, vehicle-market fit, profit after costs, and risk. A car can be profitable in one country and not profitable in another. For example, a RAV4 may be attractive in one market but not recommended for Nigeria if demand, import costs, taxes, resale price, parts availability, or competition make the margin too low.

Component	Recommendation for this project
Business problem	Help exporters, dealers, brokers, importers, diaspora buyers, and resellers decide which cars are worth exporting.
Input data	Car make, model, year, mileage, condition, Canadian price, destination country, resale price, shipping cost, fees, taxes, exchange rate.
Output	Opportunity score, estimated profit, risk level, recommended/not recommended decision, and LLM explanation.
Core technique	Structured data analysis, profit calculation, scoring, ranking.
LLM?	Yes, to explain the business recommendation.
RAG?	Only for import/export regulations, customs rules, inspection requirements, and country-specific documents.
ML model?	Optional if enough data exists for price prediction or demand prediction. Start with formulas and scoring.

6.1 Questions the system should answer

- Who is the buyer: dealer, broker, importer, diaspora buyer, individual customer, or local reseller?
- Which country or market is targeted: Nigeria, Ghana, Senegal, Cameroon, Morocco, UAE, or another specific market?
- Is this specific car a good fit for this specific market?
- What is the estimated profit after purchase price, shipping, customs, inspection, taxes, and exchange rate?
- What are the main risks: high mileage, old model year, weak demand, import restrictions, shipping delays, or low resale value?
- Which car should be prioritized for export?
- Which destination market is better for this car?
- Why is this car recommended or rejected?
- What buyer profile is best for this vehicle?
- What documents or rules must be checked before export?

6.2 Steps for Eraldo

39. **Define the business problem** - Decide which car should be exported, to which country, for which buyer, with what expected profit and risk.
40. **Define buyer profiles** - Identify dealer, importer, broker, diaspora buyer, individual buyer, or reseller priorities.
41. **Select target markets** - Choose specific countries. Do not say Africa in general.
42. **Build a small vehicle dataset** - Start with 20 to 50 cars including make, model, year, mileage, condition, Canada price, and location.
43. **Add export cost variables** - Estimate shipping, customs, duties, inspection, taxes, fees, and exchange rate.
44. **Calculate estimated profit** - estimated profit = target market resale price - Canadian purchase price - export costs.
45. **Create an opportunity score** - Rank by profit margin, mileage, age, demand, resale value, parts availability, and import risk.
46. **Use LLM for explanation** - Explain why a car is recommended or rejected in business language.
47. **Use RAG only for regulations** - Retrieve customs, import, inspection, and country-specific rules if documents are included.
48. **Build a dashboard** - Let the user select a car and country, then show profit, risk, ranking, and explanation.

6.3 Dataset options

Option	Description	When to use
Manual sample dataset	Create 20 to 50 cars manually with realistic fields.	Best for Sprint 2. Fast and simple.
Synthetic dataset	Generate fake but realistic prices, costs, countries, and resale values.	Useful to prove the logic when real data is missing.
Real car listings dataset	Collect real public listing data if allowed by terms of use.	More realistic but takes more time.
Hybrid dataset	Use real Canadian listings plus simulated export costs and resale prices.	Best balance of realism and feasibility.
RAG document set	Add regulations, customs rules, import requirements, and inspection rules.	Only for document-based questions, not profit calculation.

6.4 Sample data

car_id	make	model	year	mileage	canada_price	destination	resale_price	export_costs	profit	risk	recommendation
1	Toyota	RAV4	2021	72000	28500	Ghana	36500	6300	1700	Medium	Recommended
2	Honda	CR-V	2020	95000	24500	Nigeria	29500	6900	-1900	High	Not recommended
3	Toyota	Corolla	2019	85000	18500	Senegal	26000	5700	1800	Low	Recommended

Final pipeline: car listings -> target market data -> export cost estimation -> profit calculation -> risk score -> LLM explanation -> optional RAG for regulations

7. AI Business Assistant / WhatsApp Ordering System - Group Not Specified

Main question: Is this only a chatbot, or a transactional business assistant that manages real business actions?

This project is an AI business assistant with configurable capabilities: Receptionist FAQ, Appointment Booking, B2C Ordering, B2B Quotation, Customer Support, and Lead Generation. The LLM should understand customer messages, but the database and backend logic must control products, prices, carts, totals, orders, payments, delivery, appointment slots, and confirmations.

Component	Recommendation for this project
Business problem	Automate customer interaction for businesses through chat, ordering, FAQ, support, booking, and lead generation.
Input data	Customer messages, business configuration, menu/products, prices, cart state, policies, FAQs, contacts, order history.
Output	Assistant reply, detected intent, updated cart, calculated total, confirmed order, support ticket, lead, appointment, or translated message.
Core technique	LLM for intent and language; database and backend logic for transactions.
LLM?	Yes, essential for natural language, intent detection, multilingual support, and spelling correction.
RAG?	Yes only for FAQ, policies, support documents, refund rules, delivery rules, or business knowledge.
ML model?	Not required at first.
Database?	Essential. The database is the source of truth for menu, prices, products, and order state.

7.1 Input and output

Layer	Examples
Input data	Customer message, business settings, menu items, prices, FAQ documents, policies, contacts, cart state, order history, language.
Processing	LLM detects intent, database provides truth, backend updates orders, RAG retrieves documents when needed.
Output	Assistant reply, detected intent, updated cart, total, translated response, FAQ answer, confirmed order, support ticket, or lead.

7.2 Multilingual and misspelled products

The customer can speak in any language. The LLM detects language, normalizes spelling mistakes, translates if necessary, and maps the request to real database items. The LLM must not invent products or prices. If uncertain, it should ask a clarification question.

couche de bébé = baby diaper / diaper
 couches de bébé = baby diapers / diapers
 couches = diapers
 pampers = diapers
 bière = beer
 beer = beer

Customer: Je veux 2 couches de bébé et 1 bière.

LLM interpretation: add products to cart; diapers quantity 2; beer quantity 1.

Database matching: diapers -> real product with size options; beer -> real product with age verification if required.

7.3 Unsatisfied customer example: LLM + RAG + backend

If a customer says the food was cold and one Pepsi was missing, the LLM detects the complaint, RAG retrieves the support or refund policy, and the backend checks the real order and creates a support ticket. This is a correct combined use of LLM, RAG, and backend logic.

7.4 Steps for the team

49. **Define capabilities** - List which modules are enabled for each business: FAQ, ordering, booking, quotation, support, lead generation.
50. **Design the database** - Create tables for businesses, menus/products, prices, conversations, carts, orders, contacts, leads, and settings.

51. **Build intent detection** - Use the LLM to classify messages such as add item, remove item, ask FAQ, checkout, book appointment, or create lead.
52. **Control transactions with backend logic** - Calculate totals, manage cart state, validate prices, and confirm orders in code.
53. **Use RAG only for documents** - Retrieve FAQ, policies, support rules, delivery conditions, or refund rules.
54. **Add multilingual support** - Let the LLM understand English, French, Arabic, Spanish, or spelling mistakes.
55. **Add clarification handling** - Ask questions when product matching is uncertain.
56. **Build a demo workflow** - Show a complete order from first message to final confirmation.

Final pipeline: customer message -> LLM intent detection -> database lookup -> backend action -> optional RAG for documents -> assistant response

8. Local AI with Private Documents / Dental Office Use Case - Group Not Specified

Main question: What private documents will the local AI system use, and how will it prove that answers come from those documents?

This project is a strong RAG use case if the system uses private documents stored locally: clinic policies, dental procedures, consent forms, insurance instructions, appointment guidelines, administrative procedures, and anonymized notes. The goal is not to calculate from structured data; the goal is to answer questions from private documents without sending sensitive data to the cloud.

Component	Recommendation for this project
Business problem	Help a clinic or private office answer staff questions from local documents safely.
Input data	PDFs, Word files, consent forms, internal procedures, insurance instructions, appointment rules, policies.
Output	Answer with source document, page, section, or paragraph reference.
Core technique	Local RAG over private documents.
LLM?	Yes, local if privacy and hardware allow; cloud only if data is anonymized and allowed.
RAG?	Yes, this is a good RAG project.
ML model?	Not necessary for the core use case.

8.1 Steps

57. **Choose one niche** - Do not target everyone. Choose dental office, legal office, research assistant, or another specific use case.
58. **Collect safe documents** - Use synthetic or anonymized documents only unless there is clear authorization.
59. **Index documents locally** - Chunk the documents, create embeddings, and store them in a local vector database.
60. **Build the local chatbot** - Use a local LLM if possible, or a safe cloud setup only with anonymized content.
61. **Answer with sources** - Show document name, section, page, or paragraph used.
62. **Add refusal behavior** - If the answer is not supported by documents, say that there is not enough information.
63. **Validate answers** - Compare generated answers with the retrieved documents to reduce hallucinations.

Final pipeline: private documents -> chunking -> embeddings -> vector database -> retrieval -> LLM answer with sources -> local/private interface

9. Customer Churn Prediction - Group Not Specified

Main question: Which customer or employee is likely to leave, and why?

This project is mainly a supervised machine learning classification problem, not a RAG or document-based LLM project. The dataset contains structured variables such as tenure, contract type, monthly charges, payment method, service usage, complaints, or support interactions. The target is churn: leave or stay. An LLM can explain the prediction after the ML model, while RAG is only useful if the project includes documents such as support notes, complaints, contracts, HR notes, company policies, or legal documents.

Component	Recommendation for this project
Business problem	Predict churn risk so the company can take retention actions.
Input data	Structured customer or employee features: tenure, charges, contract, usage, complaints, support interactions.
Output	Churn = yes/no or churn probability.
Core technique	Supervised ML classification.
LLM?	Yes for explanation and business recommendations after prediction.
RAG?	Only if retrieving complaints, contracts, HR notes, legal context, policies, or support documents.
ML model?	Yes: Logistic Regression, Random Forest, XGBoost, LightGBM.

9.1 Legal or HR documents

If legal or HR issues are stored as documents, RAG can be useful to retrieve and summarize the relevant context. However, the churn prediction itself should still be done by a machine learning model using structured features. Legal and HR information is sensitive and must be handled carefully, ethically, and only with proper authorization.

Funny explanation: RAG can read the legal drama - employee complaints, company conflicts, HR notes, and Quebec labour standards - but the ML model predicts who is about to leave the movie.

9.2 Steps

64. **Define the target** - Churn yes/no or stay/leave.
65. **Select features** - Use tenure, charges, contract, service usage, complaints, support interactions, and other structured indicators.
66. **Clean and encode data** - Handle missing values, categorical variables, and class imbalance.
67. **Train models** - Compare Logistic Regression, Random Forest, XGBoost, and LightGBM.
68. **Evaluate models** - Use confusion matrix, precision, recall, F1-score, ROC-AUC.
69. **Explain predictions** - Use an LLM to explain why a person is high-risk based on model features.
70. **Use RAG only if documents exist** - Retrieve support notes, complaints, policies, or legal documents as supporting context.

10. Government Funding Search Assistant - Group Not Specified

Main question: Which government funding programs match a user or business profile, and what are the eligibility rules, deadlines, and required documents?

This is a good RAG use case because funding information is often stored in official pages, PDFs, program guides, eligibility rules, forms, and deadlines. The system should not rely only on old indexed documents because funding programs change. It should use official, recent sources or APIs when available, and the LLM should summarize the retrieved information clearly.

Component	Recommendation for this project
Business problem	Help users find relevant government funding programs and understand eligibility.
Input data	Official government pages, PDFs, eligibility rules, deadlines, program descriptions, forms, and user/business profile.
Output	Ranked funding opportunities with eligibility summary, required documents, deadline, and next steps.
Core technique	RAG plus official sources/APIs; structured filters for eligibility.
LLM?	Yes, to summarize eligibility and explain next steps.
RAG?	Yes, this is a strong RAG use case.
ML model?	Not necessary at first; ranking rules and filters may be enough.

10.1 Steps

71. **Define the user profile** - Business type, location, industry, size, project type, funding need, and eligibility constraints.
72. **Collect official sources** - Use official government websites, PDFs, program guides, and APIs when available.
73. **Build retrieval** - Index documents and retrieve relevant passages for each user query.
74. **Add structured filters** - Filter by province, industry, business stage, deadline, amount, and eligibility.
75. **Generate answer with sources** - The LLM summarizes the retrieved rules and cites the source document or page.
76. **Add freshness check** - Clearly show the date of the source or retrieval when possible.
77. **Add refusal behavior** - If the system cannot find reliable official information, it should say so instead of guessing.

Final pipeline: user/business profile -> official funding sources -> RAG retrieval -> eligibility filtering -> LLM summary -> ranked funding recommendations with sources

11. Common Sprint Review Questions and Final Reminders

The following questions can be used with any project during sprint review to force clarity on the business problem, data, model choice, and deliverable.

11.1 Universal technical questions

- What exact business problem does your project solve?
- Who is the target user or customer?
- What is the input data?
- What is the output of the system?
- Is the data structured, unstructured, or both?
- Which part is handled by code, SQL, Pandas, or DuckDB?
- Which part is handled by a machine learning model?
- Which part is handled by the LLM?
- Is RAG really necessary? If yes, what documents are retrieved?
- How will you prove that the result is correct?
- What will be your concrete Sprint 2 deliverable?
- Can you show one full workflow: input -> processing -> output?

11.2 RAG decision checklist

Question	If yes	If no
Do you have documents, PDFs, policies, manuals, reports, or long text sources?	RAG may be useful.	RAG is probably not needed.
Does the answer require exact calculations?	Use SQL/Pandas/code first.	LLM/RAG may help with explanation.
Does the answer depend on current or changing rules?	Use official/live sources or RAG with freshness checks.	Static data may be enough.
Does the system need to prove sources?	RAG with citations/sources is useful.	A normal LLM explanation may be enough.
Is the data structured rows and columns?	Use SQL/Pandas/ML/recommender algorithms.	Consider RAG only for text documents.

11.3 Final thoughts

- The LLM is good at language, but the database is the source of truth.
- RAG is useful for documents, not for exact calculations over structured data.
- Machine learning predicts from labeled data; the LLM explains the prediction.
- SQL/Pandas/DuckDB calculate exact numbers; the LLM should not invent totals.
- Fine-tuning teaches style or behavior, not fresh facts or live regulations.
- Cloud LLM solves hardware limitations, but raw sensitive data should not be sent to the cloud.
- A strong final project has a clear input, clear processing, clear output, and a working demo.

11.4 Closing message

Thank you to all the students for the presentations and discussions. The project ideas are strong and evolving in the right direction. The next priority is to make each project technically precise: define the business problem, the data, the output, the algorithm, and the role of the LLM or RAG. A good final project should not only sound innovative; it should clearly show how the system receives data, processes it correctly, and produces a useful result for a real user.